300241757 - Long Homework 1 Written Answers

Note: This document includes my written answers to the homework, but all of its contents can also be found in the file longHW1.ipynb. I've realy just copy+pasted my answers here to make it easier to read without Python code blocks in the way.

1. Training Error and New-Data Error for a Constant Predictor

Let $Y_1, ..., Y_n$ be independent observations with mean μ and variance σ^2. Ignore any predictors and fit the constant prediction rule: $\hat{f}(x) = \bar{Y} = 1/n \sum_{i=1}^{n} Y_i$

Defining the training error: $Train = 1/n \sum_{i=1}^{n} (Y_i - \bar{Y})^2$

Then let $New = (Y_{new} - \bar{Y})^2$ where Y_{new} is a new observation from the same population.

(a) Show that $\mathbb{E}[Train] = \frac{n-1}{n}\sigma^2$

$\mathbb{E}[Train] = \mathbb{E}[1/n \sum_{i=1}^{n} (Y_i - \bar{Y})^2] = 1/n \mathbb{E}[\sum_{i=1}^{n} (Y_i^2 - 2Y_i\bar{Y} + \bar{Y}^2)]$

$= 1/n (\mathbb{E}[\sum_{i=1}^{n} Y_i^2] - 2\mathbb{E}[\sum_{i=1}^{n} \bar{Y} Y_i] + \mathbb{E}[\sum_{i=1}^{n} \bar{Y}^2])$

$= 1/n (\mathbb{E}[\sum_{i=1}^{n} Y_i^2] - 2\mathbb{E}[\bar{Y} \sum_{i=1}^{n} Y_i] + n\mathbb{E}[\bar{Y}^2])$

Where $\sum_{i=1}^{n} Y_i = n\bar{Y}$ from the definition of sample mean, so:

$= 1/n (\mathbb{E}[\sum_{i=1}^{n} Y_i^2] - 2n\mathbb{E}[\bar{Y}^2] + n\mathbb{E}[\bar{Y}^2])$

$= 1/n (\mathbb{E}[\sum_{i=1}^{n} Y_i^2] - n\mathbb{E}[\bar{Y}^2])$

$= 1/n \sum_{i=1}^{n} \mathbb{E}[Y_i^2] - \mathbb{E}[\bar{Y}^2]$

Now since $\mathbb{E}[Y_i^2] = Var[Y_i]^2 + \mathbb{E}[Y_i]^2 = Var[Y]^2 + \mathbb{E}[Y]^2 = \sigma^2 + \mu^2$

$\mathbb{E}[Train] = 1/n \sum_{i=1}^{n} (\sigma^2 + \mu^2) - \mathbb{E}[\bar{Y}^2]$

$= (\sigma^2 + \mu^2) - \mathbb{E}[\bar{Y}^2]$

And because $Var[\bar{Y}] = \sigma^2/n$ and $\mathbb{E}[\bar{Y}] = \mu$ gives us $\mathbb{E}[\bar{Y}^2] = \sigma^2/n + \mu^2$, we have:

$\mathbb{E}[Train] = (\sigma^2 + \mu^2) - (\sigma^2/n + \mu^2)$

$= \sigma^2 - \sigma^2/n = \frac{n-1}{n}\sigma^2$

(b) Show that $\mathbb{E}[New] = \frac{n+1}{n}\sigma^2$

$\mathbb{E}[New] = \mathbb{E}[(Y_{new} - \bar{Y})^2] = \mathbb{E}[Y_{new}^2] - 2\mathbb{E}[Y_{New}\bar{Y}] + \mathbb{E}[\bar{Y}^2]$

Where $\mathbb{E}[Y_{new}^2] = \sigma^2 + \mu^2$, $\mathbb{E}[Y_{new}\bar{Y}] = \mathbb{E}[Y_{new}]\mathbb{E}[\bar{Y}] = \mu^2$ (since Y_new is ind. of the original sample mean) and $\mathbb{E}[\bar{Y}^2] = \sigma^2/n + \mu^2$

So $\mathbb{E}[New] = (\sigma^2 + \mu^2) - 2(\mu^2) + (\sigma^2/n + \mu^2) = \sigma^2 + \sigma^2/n = \frac{n+1}{n}\sigma^2$

(c) Explain why the expected training error is smaller, even though there is no question of choosing among complicated models

The expected training error is smaller because the sample mean $\bar{Y}$ is calculated using the same observations on which the error is evaluated. This creates an optimistically biased estimator of error. On the other hand, the new observation is independent of this predictor $\bar{Y}$, so it does not benefit from the same process.

(d) Verify the two formulas by simulation for n = 10, μ = 0, and σ = 1. Use 20,000 repetitions and initialize the random-number generator once with seed 3373.

See the code in longHW1.ipynb. Printed output was: Simulated E[Train]: 0.9045227400270848 Theoretical E[Train]: 0.9 Simulated E[New]: 1.099274517067419 Theoretical E[New]: 1.1

---

2. Empirical Study: KNN for the Iris Data

(a) Using only the development data, use five-fold stratified cross-validation to compare standardized KNN classifiers for k ∈ {1, 2, . . . , 15}. Shuffle the folds and use random_state=3373. Plot the mean cross-validation misclassification error against k.

See the code in longHW1.ipynb. Mininmum was k=6.

(b) Select k using the cross-validation results. Refit this classifier using all of the development data, then report its final test error and confusion matrix.

See the code in longHW1.ipynb. Printed output was: Test error: 0.06666666666666665 Confusion matrix for ['Setosa' 'Versicolor' 'Virginica'] [[25 0 0] [0 24 1] [0 4 21]]

(c) Fit standardized logistic regression using the same development data. Report its final test error and confusion matrix.

See the code in longHW1.ipynb. Printed output was: Test error: 0.06666666666666665 Confusion matrix for ['Setosa' 'Versicolor' 'Virginica'] [[25 0 0] [0 23 2] [0 3 22]]

(d) Comment briefly on the two classifiers: if you obtained a new independent test set from the same population, would you expect its error to be systematically larger, systematically smaller, or about the same as the observed final test error?

I would expect the new test errors to be about the same as the observed final test error for both classifiers. This is because both classifiers were only ever trained on the development dataset, meaning the final test dataset from which we produced our test errors already represented a new independent set compared to what the models were trained on.

Therefore assuming we had another test set from the same population, there is no reason to think the error would be systematically larger or smaller. This is why we split the observations into a development set and a test set in the first place.

(e) Suppose instead that you had selected k by finding the smallest error on the final test set. Explain why reporting that same minimum as final performance would tend to be optimistic.

By selecting the value of k that minimizes the error on the final test set, our choice of model would no longer be independent of that test set. Therefore the calculated minimum test error based on that set would tend to be an overly optimistic estimate of the model's true error, since we would be selecting a value of k that may have benefited from random noise specific to that test set. In this case, if we evaluated this model on a new independent test set drawn from the same population, we would expect its error to be systematically larger on average than the original error.

---

3. A Nonlinear Boundary Made Simple by One New Feature

The files circular_train.csv and circular_test.csv contain two predictors, X1, X2, and a binary response Y. The test file is a genuinely untouched final test set.

(a) Plot the training observations and describe the apparent decision boundary

See the code in longHW1.ipynb. Observations: The true decision boundary appears to be the unit circle centred at (0,0).

(b) Fit ordinary logistic regression using (X1, X2). Display its fitted decision boundary. If the boundary does not cross the plotted range, report its equation.

See code in longHW1.ipynb. Printed output was: Intercept: 1.1065303935096664 Coefficients: [-0.01672142 0.00030541]

Observations: the decision boundary does cross my plotted range, but only because I expanded the plotted range to an absurd degree, well beyond any of the values in the dataset. Indeed, under this model, every (X1,X2) pair in the original training dataset would be predicted to correspond to a 1, and the decision boundary would be well off-screen if we restricted the plot to the values in the original dataset.

The equation of the decision boundary is: $1.1065303935096664 - 0.01672142X_1 + 0.00030541X_2 = 0$

(c) Define the new feature $R^2 = X_1^2 + X_2^2$, fit logistic regression using R^2 as its predictor, and display the fitted boundary in the original (X1, X2) coordinates.

See code in longHW1.ipynb. The regression now accurately fits the circular boundary.

(d) Fit a standardized KNN classifier. Choose k ∈ {1, 3, 5, . . . , 41} using five-fold stratified crossvalidation on the training data, with mean misclassification error as the criterion. Shuffle the folds and use random state=3373.

See code in longHW1.ipynb. From the plot, here k=23 is the optimal candidate given our constraints (odd numbers between 1 and 41).

(e) Compare final test error for the three fitted rules. Also compare final test log loss for the two logistic-regression rules

See code in longHW1.ipynb. Printed output was: Naive logistic regression test error: 0.2676666666666667 R^2 logistic regression test error: 0.05233333333333334 KNN test error: 0.05566666666666664

Naive logistic regression log loss: 0.5821020118621717 R^2 logistic regression log loss: 0.12655681895427084

Among the three classifiers, the R^2 logistic regression showed the lowest test error, but KNN came quite close, suggesting either one is a viable option here. Crucially, naive logistic regression tested much worse than the other two options, suggesting that it is not a good choice of classifier for this problem.

Comparing the log loss of the two logistic regression models, the naive model again performed worse, indicating that it assigned relatively low probabilities to the true classifications of many observations / high probabilities to many incorrect classifications. This is consistent with what I observed earlier about the naive regression model learning to classify every datapoint in the training set as a 1 with reasonably high confidence, regardless of the datapoint's predictors.

(f) Explain why the improvement after introducing R2 is usefully described as a change of representation. State one conclusion supported by this experiment and one broader conclusion that is not supported

A "change of representation" is a good name for this improvement as it involves representing the predictor variables in a new way in order to capture some hidden feature of the data (ideally a feature related to its true classification rule). One conclusion supported by this experiment is its potential effectiveness: by simply using the squared-distance of (X1,X2) from (0,0) rather than using X1 and X2 directly, we were able to train a much better logistic regression model without modifying the underlying algorithm at all. Something that is not supported is that this is always an effective solution, or that every transformation will be meaningful. In this case, a datapoint's distance from (0,0) seems to have been unusually consequential to its true classification, which is why this change of representation worked so well.

---

4. Lying with Linear Regression

Throughout the question, use at least 20 distinct X-values. Every chosen X-value is paired with the observed value $Y = sin(X)$ which we will fit with the usual linear regression model.

(a) Find a collection of predictors X1, . . . , Xn for which the reported 99-percent confidence interval for the slope β1 is contained in (−∞, −0.95]. Display the points and report the estimated slope and confidence interval.

See code in longHW1.ipynb. Printed output was: X Y 0 3.041593 0.099833 1 3.063815 0.077699 2 3.086037 0.055527 3 3.108259 0.033327 4 3.130482 0.011111 5 3.152704 -0.011111 6 3.174926 -0.033327 7 3.197148 -0.055527 8 3.219370 -0.077699 9 3.241593 -0.099833

Estimated slope: -0.9987947508158653 99% CI lower bound: -0.9993918029791284 99% CI upper bound: -0.9981976986526022

(b) Find a collection of predictors X1, . . . , Xn for which the reported 99-percent confidence interval for the slope β1 is contained in [0.95,∞). Display the points and report the estimated slope and confidence interval.

See code in longHW1.ipynb. Printed output was: X Y 0 -0.100000 -0.099833 1 -0.077778 -0.077699 2 -0.055556 -0.055527 3 -0.033333 -0.033327 4 -0.011111 -0.011111 5 0.011111 0.011111 6 0.033333 0.033327 7 0.055556 0.055527 8 0.077778 0.077699 9 0.100000 0.099833

Estimated slope: 0.9987947508158653 99% CI lower bound: 0.9981976986523411 99% CI upper bound: 0.9993918029793896

(c) Would part (b) be possible if [0.95, ∞) were replaced by [100, ∞)? Explain why or why not.

No. The key to understanding why is recognizing that cos, the derivative of sin, is bounded between -1 and 1.

This fact can be used to show that the slope of the secant line connecting any two points on a sin curve must also be bounded between -1 and 1 (formalized with the mean value theorem), and then showing that the slope estimator used in linear regression can be expressed as a weighted average of the slopes between every pair of observations (each one a secant line on the sin curve). Now since each individual term in this weighted average cannot exceed 1, the slope estimator itself cannot exceed 1, therefore the confidence interval (which must contain the slope estimator) cannot be entirely above 1.

---

5. Simulation Study: Post-Selection Inference

Generate m = 80 predictors and one response for n = 100 observations according to: rng = np.random.default_rng(3373)
X = rng.normal(size=(100, 80))
y = rng.normal(size=100)
There is no relationship between any predictor and the response.

(a) For each predictor, compute its sample correlation with y. Let j1, j2 be the indices with the two largest correlations in absolute value. Fit $Y = \beta0 + \beta1Xj1 + \beta2Xj2 + \epsilon$ using the same 100 obserations. Report the two slope estimates, 95-percent confidence intervals, and p-values

See the code in longHW1.ipynb. Printed output was: Slope estimates: [0.22919598 -0.23309156] 95% confidence intervals: [[0.05052979 0.40786218] [-0.41696252 -0.04922059]] P-values: [0.01246988 0.0135107]

(b) Use the first 50 observations only to select j1, j2, and use the remaining 50 observations only to fit the displayed model and carry out the two tests. Again report 95-percent confidence intervals and p-values, and comment on the difference.

Observations: Both confidence intervals for the slope estimators are substantially wider and notably contain 0, indicating that neither predictor demonstrates a significant effect on y. Furthermore, both p-values have increased by quite a lot, far beyond any conventional significance thresholds, meaning there is no evidence of a linear relationship between the predictors and y.

This makes sense since we know X and y were randomly generated, so the front-half of some observation set being highly correlated with y would not necessarily mean that the back-half of the observations would be correlated as well.

(c) Repeat the complete simulation 200 times. Initialize the random-number generator once before the repetition loop; do not reset the seed inside the loop. For each of the two procedures, estimate the proportion of repetitions in which at least one of the two reported slope p-values is below 0.05.

See the code in longHW1.ipynb. Printed output was: Proportion for procedure 1: 0.93 Proportion for procedure 2: 0.075

(d) Explain why the first procedure produces apparently significant relationships so often, and why the second procedure restores the usual null interpretation of each individual p-value. Explain why the probability that at least one of two p-values is below 0.05 is not itself generally 0.05.

Procedure 1 produces apparently significant relationships because the predictors are selected based on their unusually high (in absolute value) sample correlations with y, and then tested using the same observations. This introduces selection bias: even though the predictors are unrelated to the response in the entire population, selecting those with the strongest observed correlations makes them more likely to appear significant. Procedure 2, while also selecting predictors based on their unusually high correlations with y, used a separate subset of those predictors for testing versus selection, so the same bias was not introduced.

The reason why the proportion of statistically-significant results did not align with 5% (despite the literal definition of a significance level of 0.05 being that, under the null hypothesis, we'd expect these results 5% of the time) differs between procedure 1 and procedure 2. For procedure 1, it's because the data has been exposed to bias as explained above. For procedure 2, despite not being exposed to the same bias, it still does not show a proportion of 0.05 because we are conducting two independent hypothesis tests and counting the result if at least one is significant. I.e. we are doing P(A or B) rather than simply P(A) or simply P(B), even if both P(A) and P(B) would individually be 0.05.

6. An Exact Bias–Variance Calculation for a Local Regression Rule

For $i = 1, ..., n$ let: $X_i = i/n, Y_i = X_i + \epsilon_i$ and $\epsilon_i \sim N(0, \sigma^2)$
The regression function is $f(x) = x$. To estimate $f(0) = 0$, use the unweighted KNN rule: $\hat{f}_k(0) = 1/k \sum_{i=1}^{k} Y_i$

(a) Compute the vias and variance of $\hat{f}_k(0)$

$Bias[\hat{f}_k(0)] = \mathbb{E}[\hat{f}_k(0)] - f(0) = \mathbb{E}[\hat{f}_k(0)]$ (since $f(0) = 0$)

$= \mathbb{E}[1/k \sum_{i=1}^{k} Y_i] = 1/k \sum_{i=1}^{k} \mathbb{E}[Y_i]$

$= 1/k \sum_{i=1}^{k} \mathbb{E}[X_i + \epsilon_i] = 1/k \sum_{i=1}^{k} \mathbb{E}[i/n + \epsilon_i]$

$= 1/k \sum_{i=1}^{k} (\mathbb{E}[i/n] + \mathbb{E}[\epsilon_i])$

$= 1/k \sum_{i=1}^{k} \mathbb{E}[i/n]$ (since $\mathbb{E}[\epsilon_i] = 0$)

$= \frac{1}{kn} \sum_{i=1}^{k} \mathbb{E}[i] = \frac{1}{kn} \sum_{i=1}^{k} i$

$= \frac{1}{kn} \frac{k(k+1)}{2}$

$= \frac{k+1}{2n}$

Now for the variance: $Var[\hat{f}_k(0)] = Var[1/k \sum_{i=1}^{k} Y_i]$

$= 1/k^2 Var[\sum_{i=1}^{k} Y_i]$

$= 1/k^2 Var[\sum_{i=1}^{k} (i/n + \epsilon_i)]$

$= 1/k^2 Var[\sum_{i=1}^{k} (i/n) + \sum_{i=1}^{k} (\epsilon_i)]$

$= 1/k^2 Var[\frac{k(k+1)}{2n} + \sum_{i=1}^{k} (\epsilon_i)]$

$= 1/k^2 Var[\sum_{i=1}^{k} \epsilon_i]$ Since $\frac{k(k+1)}{2n}$ is just a constant.

Furthermore, since each epsilon is assumed to be independent, the variance of their sum is just the sum of variances:

$= 1/k^2 \sum_{i=1}^{k} Var[\epsilon_i]$

$= 1/k^2 \sum_{i=1}^{k} \sigma^2$

$= \frac{1}{k^2} k \sigma^2$

$= \frac{\sigma^2}{k}$

(b) Show that $MSE(K) = \frac{(k+1)^2}{4n^2} + \frac{\sigma^2}{k}$

Since $MSE = Bias^2 + Variance$ we can simply use our results from part (a):

$MSE(k) = (\frac{k+1}{2n})^2 + \frac{\sigma^2}{k} = \frac{(k+1)^2}{4n^2} + \frac{\sigma^2}{k}$

(c) Treat k as continuous and replace k + 1 by k. Find the resulting approximate minimizer and show that the optimal scale is k $\asymp n^{2/3}$ when σ is fixed.

Simplifying our MSE expression with $MSE(k) \approx \frac{k^2}{4n^2} + \frac{\sigma^2}{k}$ its derivative with respect to k is:
$\frac{d}{dk}MSE(k) = \frac{k}{2n^2} - \frac{\sigma^2}{k^2}$

Setting this to 0 to find local extrema, we get: $\frac{k}{2n^2} - \frac{\sigma^2}{k^2} = 0 \implies \frac{k}{2n^2} = \frac{\sigma^2}{k^2} \implies k^3 = 2\sigma^2 n^2 \implies k = (2\sigma^2)^{1/3}(n^{2/3}) \implies k \asymp n^{2/3}$

To confirm this extremum is a minimum, we can simply recognize that the second derivative $\frac{d^2}{dk^2}MSE(k) = \frac{1}{2n^2} + \frac{2\sigma^2}{k^3}$ is positive for all k>0, since it is a sum of strictly positive terms.

(d) For n = 200 and σ = 0.2, compare k ∈ {1, 5, 15, 50} using the exact formula. Verify the same pattern by simulation.

The theoretical values are approximately:
MSE(1) = 0.040025
MSE(5) = 0.008225
MSE(15) = 0.004267
MSE(50) = 0.017056

Making the optimal k in this set k=15. This matches our minimizer from (c), since if we plug the values into $(2\sigma^2)^{1/3}(n^{2/3})$ we get about 14.736.
The experimental results below support this as well, with each approximate MSE matching the theoretical values quite closely.

See code in longHW1.ipynb. Printed output was: K= 1 theoretical MSE= 0.040025000000000005 K= 1 simulated MSE= 0.040066499291876015

K= 5 theoretical MSE= 0.008225000000000001 K= 5 simulated MSE= 0.008286523495241315

K= 15 theoretical MSE= 0.004266666666666667 K= 15 simulated MSE= 0.004329763733455776

K= 50 theoretical MSE= 0.01705625 K= 50 simulated MSE= 0.01699233646708899

(e) Explain which features of this example make the calculation possible, and why it is not a universal theorem that every more flexible method must have larger variance.

The known regression function $f(x) = x$ simplified calculations quite a lot (both because it was known and because it was so simple). Additionally, the fixed, evenly-spaced placement of the predictors: $X_i = i/n$ made KNN especially easy since the k nearest neighbours are always the first k observations. Lastly, the errors being iid with common variance also helped simplify calculations.

Altogether, in this example, decreasing k increases the flexibility of the k-NN estimator while its variance increases according to σ^2/k, but this is not a universal relationship. More generally, model flexibility doesn't determine estimator variance, since the variance also depends on the estimator function, the noise distribution, the arrangement of preictors, etc.

---

7. Predicting a New Visit or Predicting a New Person?

The file data/repeated visits.csv contains five visits from each of 500 people. The columns are: • person id and visit id; • one noisy feature called signal; • six nearly constant features called fingerprint 1, . . ., fingerprint 6; • a binary response called outcome, which is the same for all visits from one person. Do not use person id or visit id as predictors.

(a) Fit standardized logistic regression and standardized 1-NN using a random row-wise 50–50 training/test split. Use train test split with test size=0.5, random state=3373, and class stratification. Report test accuracy.

See code in longHW1.ipynb. Printed output was: 1-KNN test accuracy: 0.8872 Logistic regression test accuracy: 0.6504

(b) Repeat the comparison using GroupShuffleSplit, with person id as the group label. Use one split, test size=0.5, and random state=3373. Thus all five visits from a person must be on the same side of the split.

See code in longHW1.ipynb. Printed output was: 1-KNN test accuracy: 0.5152 Logistic regression test accuracy: 0.6648

(c) Explain why the row-wise evaluation is especially favourable to 1-NN. Under the idealization that each of a test visit's four companion visits independently lands in training with probability 1/2, calculate the probability that at least one companion visit is in the training set.

When using a row-wise split, a single person's visits can be scattered across both the training and test sets. This leakage is extremely beneficial for a 1-NN model, since the nearest neighbour according to the fingerprint predictors will often be another visit by the same person, the outcome of which is highly correlated with the outcomes of the person's other visits.

By grouping the data by person ID, ensuring no individual's visits span both the training and test sets, we are forcing the model to learn a true pattern between the predictors and outcome, rather than simply identifying each person's behaviour with the outcomes of their own companion visits.

Under the given idealization that each of a test visit's four companion visits independently land in training with probability of 1/2, the probability that at least one companion visit is in the training set is:

$$1 - P(no\ companion\ visits\ in\ training) = 1 - (1/2)^4 = 1 - 1/16 = 15/16 = 0.9375$$

(d) What prediction task does the row split approximate? What prediction task does the group split approximate?

Row split approximates predicting future events for known entities, while group split approximates generalizing behaviours to new entities.

(e) Suppose the system will be used for entirely new people. Which result should be used to compare the models? Suggest one additional check before deployment.

We should use group split, separating by individuals, since the row-wise split results would give a dangerously over-optimistic accuracy by learning the behaviour of repeat individuals moreso than generalizing the predictors to anyone's behaviour.

One additional check could be group size and representation: even with group splitting by person_id, in its current implementation, certain traits (for example, relating to the tendency to have many repeat visits), might be overrepresented in the training data.

(f) Remove the six fingerprint features and repeat the two evaluations. Explain the change.

Without the six fingerprints (and without person_id", visit_id as predictors either), the only predictor remaining will be the noisy signal variable.

See code in longHW1.ipynb. Printed output was: ROW SPLIT 1-KNN test accuracy: 0.5584 Logistic regression test accuracy: 0.6376

GROUP SPLIT 1-KNN test accuracy: 0.572 Logistic regression test accuracy: 0.656

Observations: After removing the six fingerprint features, the row-wise 1-NN accuracy decreases substantially from 0.8872 to 0.5584, while its grouped accuracy increases from 0.5152 to 0.5720. Meanwhile, logistic regression's accuracy changes only slightly, decreasing from 0.6504 to 0.6376 for the row-wise split and from 0.6648 to 0.6560 for the group split.

The major decrease for row-wise 1-NN suggests that its earlier accuracvy depended heavily on the fingerprint features, which allowed it to identify visits from the same individuals and use their known outcomes.Interestingly, the improvement in grouped 1-NN accuracy suggests that the fingerprints were actually negatively affecting its predictions for new visitors, since similarity in fingerprints does not necessarily imply similarity in outcomes between different people.